

MERCHANTLENS

Technical Document

AI Representation Optimizer for E-Commerce · Kasparro Lab Hackathon

Stack: Next.js 16 · TypeScript · Tailwind CSS · Groq API · LLaMA 3.3 70B Versatile

01 System Architecture & Data Flow

MerchantLens is a stateless Next.js 16 application with three serverless API routes. There is no persistent database, no authentication layer, and no external data store — all analysis state lives in the browser for the session duration. This was a deliberate architectural choice: zero infrastructure overhead means the tool is instantly deployable and requires no backend provisioning.

Component Map

Layer	Technology	Responsibility
Frontend SPA	Next.js 16 App Router + React 19	Input collection, state management, tab/sidebar routing, export
Analysis API	POST /api/analyze	URL scraping, LLM call, JSON parse, response to client
File API	POST /api/analyze-file	Multipart upload, text extraction (PDF/DOCX/XLSX/CSV/JSON), OCR fallback
Toolkit APIs	POST /api/trust_check, /api/policy_analyze, /api/schema	Dedicated single-purpose LLM tools in the Toolkit panel
AI Engine	Groq API — LLaMA 3.3 70B Versatile	Structured JSON generation against a strict five-dimension schema

Request Flow — URL Mode

Step	What Happens	Handled By
1	User submits store URL, description, or file	InputCard.tsx (client)
2	File bytes decoded; images sent to Tesseract.js OCR; documents sent to /api/analyze-file for text extraction	analyze/page.tsx + /api/analyze-file
3	All inputs merged into a single combined string; mode set to 'description' if any text present, else 'url'	runAnalysis() in page.tsx
4	If URL-only: server fetches page HTML, strips scripts/styles/tags, slices to 15,000 chars	/api/analyze route — fetchPageText()
5	Merged content sent to Groq with strict JSON system prompt; temperature = 0.2	Groq SDK — LLaMA 3.3 70B
6	Response regex-matched for JSON block, parsed, returned as AnalysisResult	/api/analyze route
7	Client renders scored dashboard across 6 tabs; entry appended to session history	OverviewTab, IssuesTab, etc.

02 What the LLM Does vs. What Deterministic Code Handles

We drew a clear line: the LLM is responsible only for evaluation and generation tasks where judgment is required and where errors are visible and recoverable. Everything structural — routing, parsing, rendering, validation, export — is deterministic TypeScript.

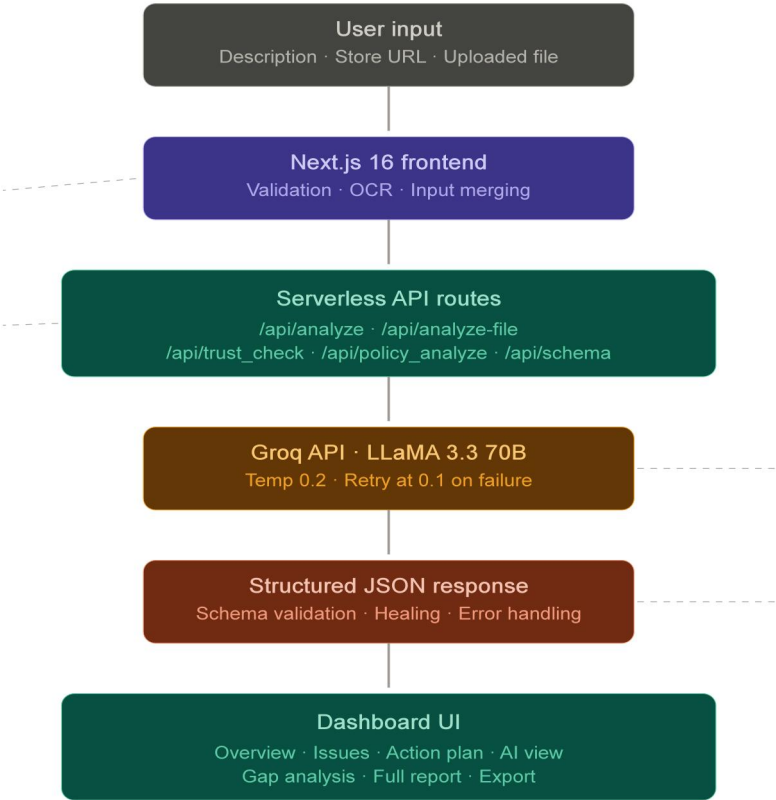
LLM Handles (Judgment Required)

- Score all five dimensions (0–100) based on inferred quality of evidence
- Generate severity-ranked issue list with root-cause descriptions
- Produce ranked action plan ordered by impact-to-effort ratio
- Write AI perception snapshot and strengths/weaknesses narrative
- Identify gaps between AI perception and inferred merchant intent
- Rewrite product descriptions in an AI-optimised format (description mode only)
- Assess input consistency across multi-source submissions

Deterministic Code Handles (No AI Needed)

- Input validation: file size, extension whitelist, character limits
- HTML scraping: regex strip of scripts, styles, tags; 15k char clip
- File text extraction: pdf-parse, mammoth, xlsx — no LLM involved
- OCR for image uploads via Tesseract.js (client-side)
- Error classification and user-friendly message mapping
- Score-to-color mapping, status chip logic, confidence derivation
- Export: Markdown build, JSON serialisation, PDF HTML generation
- History management, tab routing, theme persistence (localStorage)

Why this split? Structured prompt engineering with a strict JSON schema gives us explainable, auditable outputs at low latency (~1.5s on Groq's inference). Adding AI to routing, parsing, or export would introduce non-determinism with zero added value and significantly more failure modes.



03 Key Implementation Decisions

Strict JSON Schema Prompting

The entire analysis depends on a single Groq API call returning a valid, fully-typed JSON object. We invested heavily in the system prompt — a detailed system prompt defining exact field names, value ranges, enum constraints, and prohibition on markdown or backticks. Temperature is fixed at 0.2 across all routes to minimise hallucination drift while retaining enough variance for useful language.

The prompt explicitly states: "Do not invent policies, features, reviews, certifications, or trust signals not implied by the input. If information is missing or uncertain, be conservative and reduce scores accordingly." This prevents the model from fabricating trust signals — a real risk when the input is sparse.

Multi-Input Consistency Validation

When a user submits two or three inputs simultaneously (description + URL + file), the inputs may not belong to the same store. We address this by asking the LLM to populate an `input_consistency` object in every response — returning a `consistencyLevel` (HIGH / MEDIUM / LOW), a human-readable warning, and a list of detected mismatches. The client surfaces this as a coloured banner before the score renders. This prevents merchants from receiving a unified score built on contradictory data.

No Shopify API — URL Scraping Instead

Deep Shopify integration would require OAuth, webhook setup, and store-level permissions — shifting the entire MVP from diagnostics to auth infrastructure. Instead, the server fetches the raw HTML of any submitted URL, strips scripts/styles/comments with regex, and clips to 15,000 characters before sending to the LLM. This provides most of the analytical value while significantly reducing implementation complexity and works on any e-commerce platform, not just Shopify.

Client-Side State, No Auth, No DB

Analysis history is stored in React state (not `localStorage` or a database). This was intentional: it keeps the product fully stateless, instantly deployable to Vercel, and free of GDPR and data residency concerns in an MVP. The tradeoff is that history is lost on page reload — an acceptable limitation documented in Known Limitations below.

User-Supplied API Key Support

Every analysis route accepts an optional `apiKey` parameter. If provided, it instantiates a separate Groq client with the caller's key rather than the server default. This allows power users to use their own quota without any backend changes. The key is stored in `localStorage` under the `merchantlens_api_key` key and is never logged server-side.

04 Failure Handling & System Degradation

Failure Mode	What Triggers It	How We Handle It
LLM returns malformed JSON	Model wraps JSON in markdown, truncates output, or emits preamble text	<code>tryParseJson()</code> attempts to extract and parse the JSON block. If it fails, the system retries once — passing the bad response back as context with an explicit correction prompt at temperature 0.1. Only if the retry also fails is a 500 returned to the client. No crash, no raw dump to UI
LLM returns empty response	Groq timeout, content filter, or model error	Explicit empty-string check before any parsing. Returns 500 with actionable message. Client maps this via <code>friendlyError()</code> to a plain-English banner — the user never sees a stack trace.
URL fetch fails or times out	Site blocks bots, requires login, returns non-HTML, or DNS fails	<code>fetchPageText()</code> wraps <code>fetch</code> in <code>AbortController</code> with 12s timeout. Non-200 status, wrong content-type, or <100 chars of content each throw descriptive errors. Client

Failure Mode	What Triggers It	How We Handle It
		returns 400 with the specific reason (e.g. 'The site may block bots or require login.').
File upload fails extraction	PDF has no text layer, DOCX is corrupt, XLSX is empty	/api/analyze-file writes to a temp dir and cleans up in a finally block regardless of outcome. Empty extraction returns 400: 'No readable text found in this file.' Image files (png, jpg, jpeg, webp) never reach the server — they are handled entirely client-side via Tesseract.js OCR in extractImageText() before any API call is made. If OCR returns blank text, a user-friendly error is thrown immediately.
Rate limit or quota exceeded	429 from Groq, quota exhausted	friendlyError() in the client detects '429', 'rate limit', 'quota', 'billing' patterns in the error message and returns a specific, actionable message: 'Too many requests — please wait' or 'API quota exceeded. Check your Groq account.' Never exposes raw API error strings.
Network error / fetch fails entirely	User is offline, CORS issue, DNS failure	Caught by the outer catch block. friendlyError() matches 'failed to fetch', 'network request failed' and returns 'Network error — check your connection and try again.' Loading state is reset in the finally block so the UI does not hang.
Input is too short or blank	User clicks Analyze without filling any field	Client-side guard: canAnalyze = !loading && (input url file). Analyze button is disabled. If somehow bypassed: server returns 400 'Please provide a valid store URL or product description.' at < 3 chars.
File type or size violation	User uploads .exe, .mp4, or a file > 10 MB	Client validates extension against SUPPORTED_EXTENSIONS and size against MAX_FILE_SIZE (10 MB) before any upload occurs. Displays a dismissible red banner — no server call is made.

Error Sanitization Policy

- All errors — whether from Groq, the filesystem, or the network — are funnelled through a single friendlyError() function in page.tsx before touching the UI.
- The function pattern-matches against several known error categories (auth, rate limit, timeout, network, parse errors, server errors) and returns a fixed plain-English string.
- No raw API error messages, JSON dumps, stack traces, or HTTP status codes ever appear in the user interface.
- This applies even when the error is unexpected — the fallback is: 'Something went wrong. Please try again, or contact support if the issue persists.'

05 Known Limitations

Limitation	Root Cause	Impact
Analysis history is lost on page refresh	State stored in React useState, not localStorage or a DB	Low — users complete analyses in one session; history panel is in-session only
URL scraping fails on JavaScript-rendered stores	Server-side fetch only gets the initial HTML, not post-JS content; Shopify Liquid renders server-side so most stores work, but headless storefronts may not	Medium — affects a minority of modern headless Shopify builds

Limitation	Root Cause	Impact
LLM scores are non-deterministic	Temperature 0.2 reduces but does not eliminate variance; same input may produce slight score variation across runs	Low — scores are directional diagnostics, not precise measurements
OCR accuracy degrades on low-quality images	Tesseract.js accuracy drops on rotated, noisy, or small-font scans	Low — image upload is a secondary input path; users are prompted to paste manually if OCR fails
Toolkit FAQ Builder is static	FAQ generation uses a template, not an LLM call, to avoid extra API cost in the MVP	Low — clearly labelled; full LLM-powered FAQ builder is a planned V2 feature

06 What We Would Improve With More Time

Short-Term (Days)

- Persist analysis history to localStorage so it survives page refreshes — the data structure is already serialisable.
- Replace the static FAQ Builder with a live LLM call using the same Groq client already instantiated in the toolkit.
- Add a confidence interval display to scores — showing that a 62 is meaningfully uncertain while a 91 is robust.

Medium-Term (Weeks)

- Streaming LLM responses: use Groq's streaming API to show partial analysis results as they arrive rather than a single loading state.
- A/B scoring: run two LLM calls with slightly different prompts and return a blended score with a confidence band, making the non-determinism visible rather than hiding it.
- Webhook-triggered re-analysis: store owners could connect once and receive a daily readiness delta report as their store changes.

Architecture-Level (Longer Term)

- Persistent backend with Postgres: store analyses per-user, enable trend tracking over time, and power a merchant benchmarking leaderboard against industry verticals.
- Multi-model routing: fall back to a secondary model (Mixtral or Claude) if the primary Groq call fails or returns malformed output, rather than surfacing an error.
- Fine-tuned scoring model: accumulate labelled merchant data over time and fine-tune a smaller, faster model specifically for the five-dimension scoring rubric — reducing latency and cost.
- Real AI agent simulation: run the merchant's store URL through an actual AI shopping agent prompt and compare the agent's natural response against the merchant's stated intent — closing the loop from diagnostic to ground truth.

Design Philosophy

- We deliberately avoided multi-agent pipelines, vector databases, and fine-tuned models — not because we could not build them, but because they would not have improved the core diagnostic experience.
- A merchant who receives a score of 58/100 with three specific, ranked, plain-language fixes gets more value than one who receives an opaque agent simulation with no actionable output.
- Every architectural decision was made in service of that constraint: make the AI's judgment transparent, make the output actionable, and make the failure modes visible rather than hidden.